IFPB - Campus Campina Grande
Programação Orientada a Objetos - Checkpoint 03

Nome: Nota:

QUADRO DE RESPOSTAS

Q1: Q2: Q3: Q4: Q5:

Q6 1) 2) 3)
4) 5) 6)

Q7 1) 2)

Q8 Valor: Teste:

CENÁRIO BASE

Estadia.java

```java
public class Estadia {
  private int horas;
  private boolean encerrada;
  private Cobranca cobranca;

  public Estadia() {
    horas = 0;
    encerrada = false;
  }
  public int getHoras() {
    return horas;
  }
  public boolean isEncerrada() {
    return encerrada;
  }
  public void adicionarHoras(int qtd) {
    if (!encerrada && qtd > 0) {
      horas += qtd;
    }
  }
  public void encerrar(Cobranca opcao) {
    if (!encerrada && opcao != null) {
      cobranca = opcao;
      encerrada = true;
    }
  }
  public double calcularValor() {
    if (!encerrada) {
      return 0.0;
    }
    return cobranca.calcularValor(this);
  }
}
```

Cobranca.java

```java
public interface Cobranca {
  double calcularValor(Estadia estadia);
}
```

CobrancaPorHora.java

```java
public class CobrancaPorHora
              implements Cobranca {

  public double calcularValor(Estadia e){
    return e.getHoras() * 5.0;
  }
}
```

CobrancaPorDiaria.java

```java
public class CobrancaPorDiaria
                    implements Cobranca {

  public double calcularValor(Estadia e){
    return 40.0;
  }
}
```

Nesta avaliação, cada questão parte do cenário base; alterações valem apenas para a própria questão.

QUESTÕES

Q1. Considere uma alteração na classe `Estadia`: o parâmetro `opcao` do método `encerrar` passa a ser do tipo `CobrancaPorHora`. Os demais elementos da classe permanecem como no cenário base.

O método fica assim:

```
public void encerrar(CobrancaPorHora opcao)
{
  if (!encerrada && opcao != null) {
    cobranca = opcao;
    encerrada = true;
  }
}
```

Com essa versão do método, considere o trecho abaixo:

```
Estadia e = new Estadia();
e.adicionarHoras(3);

Cobranca c = new CobrancaPorHora();
e.encerrar(c);
```

A chamada `e.encerrar(c)` compila? Assinale a alternativa com o resultado e a justificativa corretos.

A) A chamada compila porque o objeto referenciado por `c` é da classe `CobrancaPorHora`.

B) A chamada compila porque a variável `c` e o campo `cobranca` têm tipo `Cobranca`.

C) A chamada não compila porque o tipo declarado de `c` é `Cobranca`, e o parâmetro exige `CobrancaPorHora`.

D) A chamada não compila porque uma referência de interface não pode guardar um objeto de classe concreta.

E) A chamada não compila porque o campo `cobranca` e o parâmetro `opcao` precisam ter o mesmo tipo.

Q2. O estacionamento quer oferecer a primeira hora gratuita na modalidade por hora. Até uma hora de permanência, o valor deve ser R$ 0; acima disso, devem ser cobrados R$ 5 por hora além da primeira.

A cobrança por diária deve continuar custando R$ 40. O tempo registrado em `Estadia` e informado por `getHoras` deve continuar correspondendo a todas as horas de permanência.

Qual alteração atende a esses requisitos?

A) Alterar `Estadia.adicionarHoras` para registrar uma hora a menos que a quantidade recebida.

B) Alterar `Estadia.calcularValor` para subtrair `5.0` do valor calculado pela cobrança escolhida.

C) Alterar `Estadia.getHoras` para devolver uma hora a menos que o valor registrado.

D) Alterar `Estadia.calcularValor` para devolver `0.0` até uma hora e manter o cálculo atual nas demais estadias.

E) Alterar `CobrancaPorHora.calcularValor` para aplicar a nova regra de preço.

Q3. Considere esta implementação adicional:

```
public class CobrancaEvento implements
Cobranca {
  public String getNomeEvento() {
    return "Feira";
  }

  public double calcularValor(Estadia e) {
    return 30.0;
  }
}
```

E as referências:

```
CobrancaEvento ev = new CobrancaEvento();
Cobranca mod = ev;
Estadia e = new Estadia();
e.adicionarHoras(3);
```

Qual trecho exibe o nome do evento e encerra `e` com a cobrança referenciada por `ev`, sem alterar as classes nem a interface?

A)

```
System.out.println(ev.getNomeEvento());
e.encerrar(mod);
```

B)

```
System.out.println(mod.getNomeEvento());
e.encerrar(mod);
```

C)

```
System.out.println(e.getNomeEvento());
e.encerrar(mod);
```

D)

```
System.out.println(ev.getNomeEvento());
e.encerrar(ev.getNomeEvento());
```

E)

```
System.out.println(ev.getNomeEvento());
mod.encerrar(e);
```

Q4. Um teste cria uma estadia `e` com 8 horas e executa:

```
Cobranca hora = new CobrancaPorHora();
Cobranca dia = new CobrancaPorDiaria();
Cobranca escolhida = hora;

if (dia.calcularValor(e) <
    hora.calcularValor(e)) {
    escolhida = dia;
}

e.encerrar(escolhida);
assertEquals(40.0, e.calcularValor());
```

O objetivo é escolher a modalidade mais barata. Após trocar `<` (menor que) por `>` (maior que), o teste continua passando. Qual análise explica esse resultado?

A) O teste confirma que as comparações produzem a mesma escolha para qualquer tempo.

B) Os preços são iguais nesse caso; o teste não distingue as duas comparações.

C) O método `encerrar` corrige a escolha quando recebe uma cobrança mais cara.

D) O resultado `40.0` identifica a classe do objeto escolhido como `CobrancaPorDiaria`.

E) Assumir o contrato `Cobranca` torna equivalentes os cálculos das duas modalidades.

Q5. Somente nesta questão, o método `encerrar` é substituído por:

```
public void encerrar(Cobranca escolhida) {
  if (escolhida != null) {
    cobranca = escolhida;
    encerrada = true;
  }
}
```

Qual teste passa com o método do cenário base e falha com a implementação acima? Considere os imports de JUnit corretos.

A)

```
@Test
void testeA() {
  Estadia e = new Estadia();
  e.adicionarHoras(3);
  e.encerrar(new CobrancaPorHora());
  assertEquals(15.0, e.calcularValor());
}
```

B)

```
@Test
void testeB() {
  Estadia e = new Estadia();
  e.adicionarHoras(3);
  e.encerrar(new CobrancaPorHora());
  e.encerrar(new CobrancaPorHora());
  assertEquals(15.0, e.calcularValor());
}
```

C)

```
@Test
void testeC() {
  Estadia e = new Estadia();
  e.adicionarHoras(3);
  e.encerrar(new CobrancaPorHora());
  e.encerrar(new CobrancaPorDiaria());
  assertTrue(e.isEncerrada());
}
```

D)

```
@Test
void testeD() {
  Estadia e = new Estadia();
  e.adicionarHoras(3);
  e.encerrar(new CobrancaPorHora());
  e.encerrar(new CobrancaPorDiaria());
  assertEquals(15.0, e.calcularValor());
}
```

E)

```
@Test
void testeE() {
  Estadia e = new Estadia();
  e.adicionarHoras(3);
  e.encerrar(new CobrancaPorHora());
  e.encerrar(new CobrancaPorDiaria());
  assertEquals(40.0, e.calcularValor());
}
```

Q6. Escreva os valores exibidos, na ordem das impressões.

```
Cobranca c = new CobrancaPorHora();

Estadia a = new Estadia();
a.adicionarHoras(4);
a.encerrar(c);

Estadia b = new Estadia();
b.adicionarHoras(9);

System.out.println(a.calcularValor()); //1
System.out.println(b.calcularValor()); //2
System.out.println(c.calcularValor(b)); //3
```

```java
Estadia e = new Estadia();
e.adicionarHoras(3);
e.encerrar(null);

System.out.println(e.isEncerrada()); //4

if (e.isEncerrada()) {
  e.adicionarHoras(7);
} else {
  e.adicionarHoras(2);
}

e.encerrar(new CobrancaPorHora());
System.out.println(e.calcularValor()); //5
```

```java
Estadia principal = new Estadia();
Estadia apoio = principal;

principal.adicionarHoras(6);
apoio.adicionarHoras(4);

apoio = new Estadia();
apoio.adicionarHoras(
                principal.getHoras() - 7);
principal.adicionarHoras(apoio.getHoras());

System.out.println(principal.getHoras()); //6
```

Q7. Considere esta implementação adicional:

```java
public class CobrancaAjustavel
                    implements Cobranca {
  private double tarifa = 5.0;

  public void alterarTarifa(double valor) {
    if (valor > 0) {
      tarifa = valor;
    }
  }
  public double calcularValor(Estadia e) {
      return e.getHoras() * tarifa;
  }
}
```

Escreva os valores exibidos, na ordem das impressões.

```java
CobrancaAjustavel ajuste = new
CobrancaAjustavel();
Cobranca modalidade = ajuste;

Estadia e = new Estadia();
e.adicionarHoras(7);
e.encerrar(modalidade);

double valorRegistrado = e.calcularValor();
ajuste.alterarTarifa(4.0);

System.out.println(valorRegistrado); //1
System.out.println(e.calcularValor()); //2
```

Q8. Somente nesta questão, o método `calcularValor` de `Estadia` é substituído por:

```java
public double calcularValor() {
  if (!encerrada) {
    return 0.0;
  }
  return horas * 5.0;
}
```

Considere o teste JUnit abaixo, com os imports corretos. Escreva o valor exibido pelo `println` e indique se o teste **PASSA** ou **FALHA**.

```java
@Test
void teste() {
  Estadia e = new Estadia();
  Cobranca mod = new CobrancaPorDiaria();
  e.adicionarHoras(6);

  assertFalse(e.isEncerrada());
  e.encerrar(mod);
  System.out.println(e.calcularValor());

  assertEquals(
      mod.calcularValor(e),
      e.calcularValor()
  );
}
```